

NLP MODÜLÜ

Hugging Face Ekosistemi

ve Lokal Model Kullanımı

Transformers

Safetensors

PyTorch

Quantization

Model Cards

Hugging Face Nedir?



Analoji: Hugging Face, makine öğrenmesi modelleri için **GitHub**'tir. Kod yerine **model ağırlıkları**, veri setleri ve uygulamalar paylaşılır.



Models

2M+ açık kaynak model: LLM, görüntü, ses, video. Llama, Mistral, Phi, Qwen vb.



Datasets

200K+ veri seti. NLP, görüntü, ses; doğrudan datasets API ile yüklenebilir.



Spaces

Gradio/Streamlit ile model demoları. Ücretsiz CPU, ücretli GPU host'lama.



Inference API

Modeli indirmeden REST/HTTP üzerinden sorgulama. Serverless ve dedicated.

Hub'da Filtreleme • Sol Panel Haritası

huggingface.co/models — 2M+ modeli filtrelemenin 6 yolu. [Sol paneldeki her kategori, doğru modeli bulmanın kilidi.](#)



Tasks ?pipeline_tag=...

Modelin ne işe yaradığı: text-generation, image-to-text, ASR, embedding...



Libraries ?library=...

Hangi framework: transformers, diffusers, GGUF, safetensors, ONNX...



Languages ?language=...

Modelin desteklediği dil(ler): tr, en, multilingual...



Licenses ?license=...

Apache-2.0, MIT, Llama Community, gated — ticari kullanım uygun mu?



Apps ?other=...

Hangi yerel uygulamayla uyumlu: Ollama, LM Studio, vLLM, llama.cpp...



Inference Providers ?inference_provider=...

Bulutta hangi sağlayıcıdan inference: Groq, Together AI, Cerebras...

Tasks · Modelin Görevi

pipeline_tag — her modelin tek bir birincil görevi vardır. `transformers.pipeline()` bu etiketi okur. Toplam 50+ görev kategorisi var.

Text Generation

`text-generation`

Örnek: Llama-3, Mistral, Phi-3, Qwen

Chatbot, kod, içerik üretimi

Image-Text-to-Text

`image-text-to-text`

Örnek: LLaVA, Qwen-VL, Idefics3

Görsel soru-cevap (VLM)

Text-to-Image

`text-to-image`

Örnek: FLUX.1, SDXL, SD-3.5

Prompt → görsel üretimi

Image-to-Image

`image-to-image`

Örnek: InstructPix2Pix, ControlNet

Görsel düzenleme, stil aktarımı

Text-to-Video

`text-to-video`

Örnek: CogVideoX, Hunyuan, Mochi-1

Prompt → kısa video klip

Text-to-Speech

`text-to-speech`

Örnek: Coqui XTTS, F5-TTS, Kokoro

Seslendirme, podcast üretimi

Speech Recognition

`automatic-speech-recognition`

Örnek: Whisper-v3, Distil-Whisper

Ses → metin transkripsiyon

Any-to-Any

`any-to-any`

Örnek: GPT-4o tarzı omni modeller

Çoklu modalite girdi+çıkı

 **50+ pipeline tag:** translation · summarization · NER · sentence-similarity · object-detection · zero-shot · forecasting...

Libraries • Hangi Format & Framework?

Bir model birden çok library etiketi taşıyabilir (örn. *transformers* + *pytorch* + *safetensors*). Yüklemeden önce uyumlu kütüphaneyi seçmek için bu filtre kullanılır.

Framework (motor)	Üst Seviye Kütüphane	Dosya / Çalıştırma Formatı
PyTorch Standart. Üretim & araştırma.	Transformers LLM, vision, audio — universal API.	Safetensors Güvenli tensor — modern HF default.
TensorFlow Google ekosistemi, üretim.	Diffusers Stable Diffusion, FLUX, video gen.	GGUF llama.cpp / Ollama yerel LLM.
JAX TPU + araştırma; Flax ile.	sentence-transformers Embedding & RAG için.	ONNX / MLX Cross-platform / Apple Silicon.



Yerel CPU/Mac için **GGUF**. NVIDIA GPU production için **Safetensors** + **Transformers**. Tarayıcıda **Transformers.js** + **ONNX**.

Apps & Inference Providers

> Apps · Yerel Çalıştırma

?other=...

Model kartında uyumlu uygulama etiketleri.

- **Ollama** · GGUF · CLI + REST API · en kolay
- **llama.cpp** · GGUF · C++ · CPU/Metal en hızlı
- **LM Studio** · GGUF · GUI · model keşfi + chat
- **vLLM** · Safetensors · production GPU serving
- **MLX LM** · Apple Silicon · M1-M4 native
- **Jan / Draw Things** · Privacy-first chat / SD GUI (macOS)

☁ Inference Providers · Bulut

?inference_provider=...

Donanım yok? HF üzerinden serverless API.

- **Groq** · LPU · ultra-düşük latency (~ms)
- **Cerebras** · Wafer-scale chip · 1000+ tok/s
- **Together AI** · Geniş model katalogu · esnek
- **SambaNova** · Reconfigurable dataflow chip
- **fal.ai** · Görsel/video üretim odaklı
- **Novita / Hyperbolic** · Açık kaynak LLM serving

🔒 **Apps**: kontrol, gizlilik, sıfır API maliyeti, donanım gerekli. **Providers**: hızlı başla, donanım gerekmez, kullandıkça öde.

Temel Terimler Sözlüğü

Model

Eğitilmiş sinir ağı; mimarisi + öğrenilmiş parametreler.

Weights / Parameters

Modelin öğrendiği sayısal değerler (örn. Llama-3-8B → 8 milyar parametre).

Inference

Eğitilmiş modeli kullanarak tahmin/üretim yapma süreci.

Embedding

Metnin / görüntünün anlamını temsil eden sayısal vektör.

Checkpoint

Modelin belirli bir eğitim anındaki ağırlıklar dosyası.

Tokenizer

Metni model girdisine çeviren bileşen. Her modelin kendi tokenizer'ı vardır.

Fine-tuning

Önceden eğitilmiş modeli kendi verinizle yeniden eğitme.

Context Window

Modelin tek seferde işleyebileceği token sayısı (örn. 8K, 128K, 1M).

Model Card · Neyi Okumalıyız?



Model Card

Modelin kimlik kartı. Her HF model sayfasında README.md olarak yer alır. Modeli kullanmadan önce mutlaka okunmalı.



Lisans uyumsuzluğu, dil sınırı veya bias raporu projeyi durdurabilir.

- ✓ **Model Description** — Mimari, parametre sayısı, eğitim hedefi
- ✓ **Intended Use & Limitations** — Hangi göreve uygun, ne için DEĞİL
- ✓ **Training Data** — Hangi veriyle eğitildi, kapsama dahil mi?
- ✓ **Evaluation / Benchmarks** — MMLU, HellaSwag, GLUE skorları
- ✓ **License** — Apache-2.0, MIT, Llama Community, gated, vb.
- ✓ **Bias, Risks & Limitations** — Bilinen yanlılıklar, halüsinasyon riski
- ✓ **Carbon Footprint** — Eğitim sırasındaki CO₂ emisyonu

Model Dosya Türleri

Format	Açıklama	Güvenlik	Hız	Kullanım Alanı
.bin / .pt	PyTorch pickle formatı (eski standart)	⚠ Riskli (RCE)	Orta	Eski PyTorch ckpt
.safetensors	Güvenli tensor formatı (HF standardı)	✓ Güvenli	✓ Çok hızlı	Tüm modern HF modelleri
.gguf	llama.cpp formatı, CPU/Mac M1-2 dostu	✓ Güvenli	✓ Hızlı	Yerel LLM (Ollama, LM Studio)
.onnx	Cross-framework standart	✓ Güvenli	Hızlı	Production deployment
.ckpt	Stable Diffusion eski format (pickle)	⚠ Riskli	Orta	SD 1.x ekosistemi

 **Genel kural:** Mümkün olduğunda **.safetensors** kullan. Yerel LLM için **.gguf** tercih et.

Safetensors · Neden Pickle Yerine?



.bin (Pickle) — Risk

Arbitrary Code Execution

Pickle dosyaları yüklenirken Python kodu çalıştırabilir. Kötü niyetli .bin dosyası bilgisayarınızı ele geçirebilir.

Yavaş yükleme

Tüm dosya RAM'e kopyalanır, sonra tensorlere parse edilir.

İçerik denetlenemez

Çalıştırmadan dosyanın güvenliğini kontrol edemezsiniz.



.safetensors — Çözüm

Güvenli

Yalnızca tensor verisi içerir, kod çalıştıramaz. Header okunarak içerik denetlenebilir.

Zero-copy yükleme

Memory-mapping ile diskten doğrudan GPU'ya. Pickle'a göre 2-10x hızlı.

Framework-agnostic

PyTorch, TensorFlow, JAX, NumPy — hepsi okuyabilir. HF'in resmi standardı.

PyTorch / Torch · Ne İşe Yarar?

Yığın Mimarisi

Uygulama Kodu (sizin)

transformers, diffusers

PyTorch (autograd + nn)

ATen / Tensor Library (C++)

CUDA / Metal / CPU BLAS

GPU / CPU Donanım



Tensor & GPU Hesaplama

NumPy benzeri ama GPU'da çalışır. CUDA ile 50-100x hızlanma.



Autograd

Otomatik türev hesaplama; geri yayılım (backprop) için zorunlu.



torch.nn

Hazır katmanlar: Linear, Conv2d, Transformer, LayerNorm...



Inference Motoru

transformers altında tüm forward pass'leri PyTorch yürütür.

Kütüphane Mimarisi

transformers — Hugging Face'in amiral kütüphanesi. 250K+ modeli tek tip API ile kullanmanızı sağlar.

Pipeline API

En yüksek seviye · 3 satırda inference

`pipeline('sentiment-analysis')(metin) → sonuç`. Otomatik model + tokenizer.

Auto Classes

Orta seviye · esnek kontrol

`AutoTokenizer`, `AutoModel`, `AutoModelForCausalLM` — checkpoint adından doğru sınıfı seçer.

Modeling Classes

Düşük seviye · özelleştirme

`BertModel`, `GPT2LMHeadModel`, `LlamaForCausalLM` — mimari spesifik sınıflar. Fine-tuning için.

Pipeline API · 3 Satırda Inference



```
# Sentiment analizi (Türkçe destekli model)
from transformers import pipeline
clf = pipeline("sentiment-analysis", model="savasy/bert-base-turkish-sentiment-cased")
clf("Bu eğitim harikaydı, çok şey öğrendim.")
# → [{'label': 'positive', 'score': 0.998}]
```

Yaygın pipeline görevleri

sentiment-analysis	Duygu / polarite	text-generation	LLM ile metin üretimi
summarization	Özetleme	translation_xx_to_yy	Çeviri
ner	Adlandırılmış varlık	zero-shot-classification	Etiketsiz sınıflandırma
automatic-speech-recognition	Konuşma → metin (Whisper)	image-classification	Görüntü sınıflandırma

AutoModel & AutoTokenizer · Manuel Kontrol



```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_id = "microsoft/Phi-3-mini-4k-instruct"

# 1) Tokenizer ve modeli yükle (ilk seferde indirir, sonra cache'ten)
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id, torch_dtype=torch.float16, device_map="auto")

# 2) Metni tokenize et
inputs = tokenizer("Veri biliminde overfitting nedir?", return_tensors="pt").to("cuda")

# 3) Üretim ve çıktıyı çöz (device_map='auto' GPU+CPU dağıtır)
outputs = model.generate(**inputs, max_new_tokens=200, temperature=0.7)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Kardeş Kütüphaneler · Ne Zaman Lazım?



accelerate

Multi-GPU, CPU offloading, disk offloading

Büyük modeli sınırlı RAM ile çalıştırırken (device_map='auto').



bitsandbytes

8-bit & 4-bit quantization

load_in_4bit=True ile 70B modeli 24GB GPU'da çalıştırma.



peft

Parameter-Efficient Fine-Tuning

LoRA / QLoRA ile düşük maliyetli fine-tuning. Tam ağırlık eğitmek yerine.



datasets

Veri seti yükleme & streaming

load_dataset('imdb') tek satırda. Disk dolmadan stream edebilir.



trl

RLHF, DPO, SFT trainer'ları

Modeli insan tercihleriyle hizalama (ChatGPT'nin RLHF pipeline'ı).



diffusers

Stable Diffusion & görüntü üretimi

SD, SDXL, Flux, AnimateDiff — diffusion modelleri için ayrı API.

Quantization · Hafızayı Sıkıştırmak

Quantization = model ağırlıklarının **daha az bit** ile temsil edilmesi. *Bellek azalır, hız artar — küçük doğruluk kaybı karşılığında.*

Hassasiyet	Bit/Param	Llama-3-8B Boyutu	Kalite Kaybı	Tipik Kullanım
FP32 (full)	32	~32 GB	—	Eğitim (referans)
FP16 / BF16	16	~16 GB	≈ %0	Standart inference
INT8	8	~8 GB	%1-2	Dengeli üretim
INT4 (Q4)	4	~4.5 GB	%3-7	Yerel LLM, GPU'su küçük
INT2 / Q2_K	2	~2.5 GB	%10+	Aşırı kısıtlı donanım

 **Pratik kural:** Q4_K_M genelde kalite ↔ boyut dengesinin tatlı noktasıdır.

Quantization Yöntemleri

GPTQ

GPU · transformers + auto-gptq

Post-training, layer-by-layer. NVIDIA GPU'da hızlı. 4-bit standardı.

AWQ

GPU · Activation-aware Weight Q.

GPTQ'dan biraz daha iyi kalite, benzer hız. vLLM ile uyumlu.

GGUF

CPU/Mac · llama.cpp ekosistemi

Q2_K, Q4_K_M, Q5_K_M, Q8_0 varyantları. Ollama, LM Studio kullanır.

bitsandbytes NF4

GPU · QLoRA için tasarlandı

load_in_4bit=True ile transformers'ta tek satırda. Fine-tuning dostu.

Donanıma Hangi Model Sığar?

VRAM / RAM	FP16 (tam)	Q8 (8-bit)	Q4 (4-bit)
6 GB GPU	—	Phi-3-mini	Llama-3-8B
8 GB GPU	—	Llama-3-8B	Mistral-7B, Qwen-7B
12 GB GPU	Phi-3-mini	Mistral-7B	Llama-3-13B, Mixtral-8x7B*
24 GB GPU	Llama-3-8B	Llama-3-13B	Llama-3-70B*, Qwen-32B
48 GB GPU	Llama-3-13B	Llama-3-34B	Llama-3-70B, Mixtral-8x22B
64 GB RAM (CPU, GGUF)	—	Llama-3-13B	Llama-3-70B Q4_K_M

* sınırda — context window'a göre OOM olabilir. **Kural:** Model boyutu + 2-4 GB (KV cache + activations) $\leq$ VRAM.

Yerelde Model Çalıştırma Yolları



transformers (Python)

- + En esnek, fine-tune edilebilir
- Kurulum karmaşık, GPU önerilir
- *Geliştirici, ML mühendisi*



Ollama

- + Tek komutla model çekme, REST API
- Sadece GGUF, sınırlı kontrol
- *Hızlı yerel prototip*



llama.cpp

- + C++, CPU/Mac M-serisi en hızlısı
- CLI, ayrı binary derleme
- *Düşük seviye optimizasyon*



LM Studio

- + GUI, model keşfi, chat arayüzü
- Kapalı kaynak, otomasyon zor
- *Teknik olmayan kullanıcı*



vLLM

- + Production throughput, PagedAttention
- NVIDIA GPU, server kurulumu
- *Çoklu kullanıcı API sunumu*



text-generation-webui

- + Web UI, çoklu backend, LoRA
- Bağımlılık karmaşası
- *Deneme, karşılaştırma*

Uçtan Uca Workflow

1



Model Seç

huggingface.co/models · filter: text-gen, language: tr, license: apache-2.0

2



Login (gated için)

`huggingface-cli login`
veya `HF_TOKEN` env değişkeni

3



İndir & Cache

`from_pretrained()` ilk seferde `~/.cache/huggingface/` altına çeker

4



Inference

`tokenizer` → `model.generate()` → `tokenizer.decode()`

5



Servis Et

FastAPI / Gradio / Streamlit ile sarmala. Ollama serve REST API açar.

Sık Karşılaşılan Hatalar & Çözümler

CUDA out of memory	→ Daha küçük quantization (Q8→Q4), torch_dtype=fp16, device_map='auto', max_new_tokens düşür
trust_remote_code=True uyarısı	→ Model özel kod içeriyor. Sadece güvendiğin yayıncılarda (microsoft, meta-llama, Qwen) açın.
401 / Gated repo access denied	→ huggingface.co/{model} sayfasından kabul + huggingface-cli login + access token
Disk full / cache şişti	→ ~/.cache/huggingface/hub temizle. HF_HOME env değişkeni ile başka diske yönlendir.
Tokenizer eşleşmiyor / acayip çıktı	→ Modelin tokenizer'ını AYNI repo'dan yükle. AutoTokenizer.from_pretrained(model_id)
Çıktı tekrar ediyor / kalitesiz	→ temperature=0.7, top_p=0.9, repetition_penalty=1.1; chat modellerinde apply_chat_template kullan

Bir Modeli Kullanmanın 3 Yolu

Hugging Face modeli kullanmanın 3 ana yolu vardır. Donanımınız, gizlilik ihtiyacınız ve kontrol ihtiyacınız hangi yöntemi seçeceğinizi belirler.

1

Inference API

Uzak Sunucu



- Model indirilmez
- API key ile çağrı
- Kullandıkça öde / kotalı
- Donanım gerekmez

✓ Hızlı prototip, demo, düşük hacim

✗ Veri gizliliği, yüksek hacim, internet bağımlılığı

2

pipeline()

Yerel · Yüksek Seviye



- Model otomatik indirilir
- 3 satır kod
- Yerel çalışır (cache'lenir)
- İlk indirme: ~MB'tan ~GB'a

✓ Yerel demo, eğitim, hızlı entegrasyon

✗ Düşük seviye kontrol yok, fine-tune zor

3

AutoModel

Yerel · Manuel



- Tokenizer & model ayrı
- Tensor seviyesinde kontrol
- Batch, quantization, device_map
- Fine-tuning'e uygun

✓ Production, fine-tuning, optimizasyon

✗ Daha çok kod, ML bilgisi gerekir

HF Token · Alma ve Kullanma

Token nasıl alınır?

- 1 huggingface.co/join → ücretsiz hesap
- 2 Settings → Access Tokens
- 3 New token → Type: Read (veya Fine-grained)
- 4 hf_xxxx... kopyala — TEKRAR GÖRÜLEMEZ



Token tipleri

- **Read:** model indir, inference yap (en yaygın)
- **Write:** model/dataset yükle
- **Fine-grained:** spesifik repo/scope (en güvenli)

Token nasıl kullanılır?

```
# Colab Secrets (önerilen)
from google.colab import userdata
import os
os.environ["HF_TOKEN"] = \
    userdata.get("HF_TOKEN")

# Yerel CLI
# huggingface-cli login
```



Ücretsiz Plan Limitleri

- **Model indirme:** sınırsız (rate limited)
- **Inference API:** aylık kredi tabanlı kota
- **Gated modeller:** Llama, Gemma onay şart
- **PRO (\$9/ay):** 20× kota, ZeroGPU Spaces

Inference API · Uzak Sunucu

Model bilgisayarınıza inmez. *InferenceClient* HF üzerinden çağrı yapar. *provider* parametresiyle Groq, Together AI, Cerebras seçilebilir.



```
# Kurulum: pip install -q huggingface_hub
from huggingface_hub import InferenceClient

# A) Doğrudan HF üzerinden (varsayılan provider)
client = InferenceClient(token=os.environ["HF_TOKEN"])
resp = client.chat.completions.create(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[{"role": "user", "content": "NLP nedir?"}]
)
print(resp.choices[0].message.content)

# B) Spesifik provider (Groq – ultra hızlı)
client = InferenceClient(provider="groq", token=os.environ["HF_TOKEN"])
# Provider'lar OpenAI-uyumlu API döner; aynı kod farklı backend.
```

 **Avantaj:** donanım yok, 1 satırda 70B model. **Dezavantaj:** veri dışarı çıkar, kota var, internet şart.

pipeline() · Yerel Yüksek Seviye

Model otomatik indirilir (~/.cache/huggingface/) ve cache'lenir. Sonraki çağrılar disk'ten okur. 3 satırda inference.



```
# Kurulum: pip install -q transformers torch
from transformers import pipeline

# A) Türkçe duygu analizi (~440 MB, ilk çağrıda indirir)
clf = pipeline("sentiment-analysis",
               model="savasy/bert-base-turkish-sentiment-cased")
print(clf("Bu eğitim çok faydalıydı, tavsiye ederim."))
# → [{'label': 'positive', 'score': 0.998}]

# B) LLM ile metin üretimi (yerel GPU/CPU)
gen = pipeline("text-generation", model="microsoft/Phi-3-mini-4k-instruct",
               device_map="auto", torch_dtype="auto")
out = gen("Overfitting nedir?", max_new_tokens=150)
print(out[0]["generated_text"])
```

💡 **Avantaj:** yerel, gizli, internet sonrası bağımsız. **Dezavantaj:** disk + RAM, ilk indirme bekleme, fine-tune kısıtlı.

AutoModel & Tokenizer · Manuel Kontrol

Tokenizer ve model ayrı yüklenir. Tensor seviyesinde kontrol → batch, *quantization*, fine-tuning. Production tercihi.

```
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

model_id = "microsoft/Phi-3-mini-4k-instruct"

# 4-bit quantization (12 GB → ~2.5 GB)
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_compute_dtype=torch.float16)
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id, quantization_config=bnb, device_map="auto")

# Chat template (Phi-3 doğru formatlama bekler)
msgs = [{"role": "user", "content": "Embedding nedir?"}]
inputs = tokenizer.apply_chat_template(msgs, return_tensors="pt").to(model.device)
out = model.generate(inputs, max_new_tokens=200, temperature=0.7)
print(tokenizer.decode(out[0], skip_special_tokens=True))
```



Avantaj: tam kontrol, quantization, fine-tune, batch. **Dezavantaj:** daha çok kod, doğru tokenizer/template şart.

Hangi Yöntemi Ne Zaman?

Kriter	Inference API	pipeline()	AutoModel
Donanım gerek?	Hayır	Evet (CPU/GPU)	Evet (GPU önerilir)
Kurulum süresi	< 1 dk	Model boyutu kadar	Model boyutu kadar
Veri gizliliği	⚠ Dışarı çıkar	✓ Yerel	✓ Yerel
İlk inference hızı	Çok hızlı	İndirme + warm-up	İndirme + warm-up
Sürekli kullanım maliyeti	Kota / \$	Bedava	Bedava
Fine-tuning	✗ Çoğunda yok	Sınırlı	✓ Tam destek
Kod karmaşıklığı	Düşük	Çok düşük	Orta-yüksek
İdeal kullanım	Demo, düşük hacim	Eğitim, prototip	Production, fine-tune



Hızlı karar: Sınıfta gösterim → **API** · Hızlı yerel demo → **pipeline** · Capstone / üretim → **AutoModel**

Şimdi Sıra Sizde

Atölye Görevi

Aynı modeli 3 farklı yöntemle çalıştır, sonuçları karşılaştır:

1. **Inference API** — HF token al, InferenceClient ile Llama-3.1-8B'ye 1 prompt gönder
2. **pipeline()** — Phi-3-mini'yi Colab'da yerel indir, Türkçe prompt ile üret
3. **AutoModel + 4-bit** — bitsandbytes ile aynı Phi-3'ü 4-bit yükle, hız farkını ölç

Kaynaklar

- ▶ huggingface.co/docs/transformers — resmi dokümantasyon
- ▶ huggingface.co/learn/nlp-course — ücretsiz HF kursu
- ▶ github.com/huggingface/transformers — kaynak kod & örnekler
- ▶ ollama.com/library — hazır GGUF modeller